

Introduction to Intensive Programming in C++

Lecture 1: Introduction

Elias TSIGARIDAS

École Polytechnique

Contents

General Info

Search in an unsorted array

Search in a sorted array

Basic algebraic operations

Multiplication

Exponentiation

Fibonacci

Additional issues

General info :: who



Gleb POGUDIN

gleb.pogudin@polytechnique.fr

(TD)

Elias TSIGARIDAS

elias.tsigaridas@inria.fr

(Lectures)

General info :: what

What ?

- Given a problem, we want to
 - understand it
 - solve it (using efficient algorithms and mathematical insight)
 - translate the algorithms to efficient code (mainly C++)
 - solve it as quickly as possible (not really!), and
 - implement the solution correctly (without bugs! as much as possible)
- This course goes along these lines.
- It has the additional ambition to give you motivation to go further with Algorithms and Programming.
- For any solution, you should always ask:
Can we do it better? Is there a better/faster/more elegant solution?

How?

- Lectures
 - (brief) presentation of (common) algorithmic techniques
 - (some) programming tricks
 - notions of complexity analysis and bounds
 - quiz questions to have some interaction
- TD
 - hands on experience
 - coding exercises that follow the lectures
 - practice (there is really **no** alternative to this)



Did you ever write a program in C++?

Where ?

- Lectures + TD

Turing building:

Nicolaas Govert de Bruinj

- Exams

3-5 coding problems (details later)

Some questions on your code, as well

Evaluation

Exams + TD

- **Programming language(s)**

We focus on C++.

The course is a good way to learn the *fundamentals* of C++.

To master a programming language takes (IMHO a lot of) time

- **Environment:** e.g., g++, clang, Emacs, VScode (up to you really)

- **Additional resources:**

Christoph Dürr's book (poly)

Skiena, Revilla: "Programming Challenges"

Ask us for additional references.

Questions?

Contents

General Info

Search in an unsorted array

Search in a sorted array

Basic algebraic operations

Multiplication

Exponentiation

Fibonacci

Additional issues

Find the maximum element in an array

```
#include <iostream>
#include <limits>
using namespace std;

int A[numeric_limits<int>::max()];

int main() {
    int n;
    cin >> n;
    for (int i = 0; i < n; ++i) { cin >> A[i];}

    int M = numeric_limits<int>::min();
    for (int i = 0; i < n; ++i) {
        if (M < A[i]) { M = A[i]; }
    }

    cout << M << endl;
    return 0;
}
```

Find the maximum element in an array

reserve the memory
at compile time

```
#include <iostream>
#include <limits>
using namespace std;
```

4 bytes

2147483647

```
int A[numeric_limits<int>::max()];
```

```
int main() {
```

```
    int n;
```

```
    cin >> n;
```

```
    for (int i = 0; i < n; ++i) { cin >> A[i]; }
```

```
    int M = numeric_limits<int>::min();
```

```
    for (int i = 0; i < n; ++i) {
```

```
        if (M < A[i]) { M = A[i]; }
```

```
    }
```

```
    cout << M << endl;
```

```
    return 0;
```

```
}
```

Input : Array of size n

Output : max elem in A

A =

3	8	2	5	1	4	7	6	3
---	---	---	---	---	---	---	---	---

i=0

M = -∞

Complexity : O(n) ops
comparisons

Space : O(1)
we traverse the matrix once

Find the maximum element in an array (à la STL)

STL : Standard Template Library

basic data structures
algorithms
templates

```
#include <iostream>
#include <limits>
#include <algorithm>
using namespace std;
```

```
int A[numeric_limits<int>::max()];
```

```
int main() {
    int n;
    cin >> n;
    for (int i = 0; i < n; ++i) { cin >> A[i]; }
```

```
    cout << *max_element(A, A+n) << endl;
    return 0;
```

```
}
```

← find the max element

Q: Can we use `std::vector` ?

What can go wrong?

Some common mistakes

- no initialization of variables
- use 32-bit integers instead of 64-bit *long int, int64_t, long double etc*
- use unsigned int instead of signed int
- out-of-bounds array indices
- wrong use of pointers

Learn everything about them, but do not use them (C++14)

- avoid macros: in modern C++ use "const int"
- take care of the boundary cases
- use "break" and "next" wisely

⑧ *Read the code of good "programmers" !*



What is difference between "#define N 10" and "const int N=10;"

we can redefine it

no redefinition

makes the value of an identifier const. It has a scope 10

Find the 2 largest elements in an array

```
#include <iostream>
#include <limits>
using namespace std;

int A[numeric_limits<int>::max()];

int main() {
    int n;
    cin >> n;
    for (int i = 0; i < n; ++i) { cin >> A[i];}

    int M2 = numeric_limits<int>::min();
    int M1 = M2 + 1;
    for (int i = 0; i < n; ++i) {
        if (M1 < A[i]) { M2 = M1; M1 = A[i]; }
        else if (M2 < A[i]) { M2 = A[i]; }
    }
    cout << M1 << " " << M2 << endl;
    return 0;
}
```

Complexity $O(2n) = O(n)$
#comparisons

k-largest elements
in $O(kn)$
#comparisons

Find the k -th smallest element in an array

The Partition function [Hoare: 1961]

```
PARTITION( $A[1..n], p$ ):  
  swap  $A[p] \leftrightarrow A[n]$   
   $\ell \leftarrow 0$  ⟨⟨#items < pivot⟩⟩  
  for  $i \leftarrow 1$  to  $n-1$   
    if  $A[i] < A[n]$   
       $\ell \leftarrow \ell + 1$   
      swap  $A[\ell] \leftrightarrow A[i]$   
  swap  $A[n] \leftrightarrow A[\ell + 1]$   
  return  $\ell + 1$ 
```



smaller

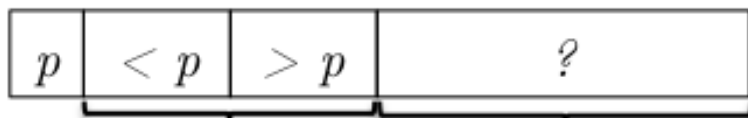
bigger

it does not necessarily
keep the order



Complexity: $O(n)$

Easy implementation
using an extra array



already partitioned unpartitioned

We can implement the algorithm
IN PLACE?
(no extra memory)

Find the k -th smallest element in an array

The Partition function [Hoare: 1961]

```
PARTITION( $A[1..n], p$ ):  
  swap  $A[p] \leftrightarrow A[n]$   
   $\ell \leftarrow 0$  ⟨⟨#items < pivot⟩⟩  
  for  $i \leftarrow 1$  to  $n-1$   
    if  $A[i] < A[n]$   
       $\ell \leftarrow \ell + 1$   
      swap  $A[\ell] \leftrightarrow A[i]$   
  swap  $A[n] \leftrightarrow A[\ell + 1]$   
  return  $\ell + 1$ 
```

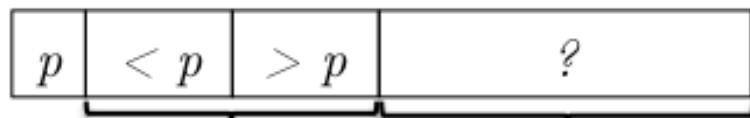
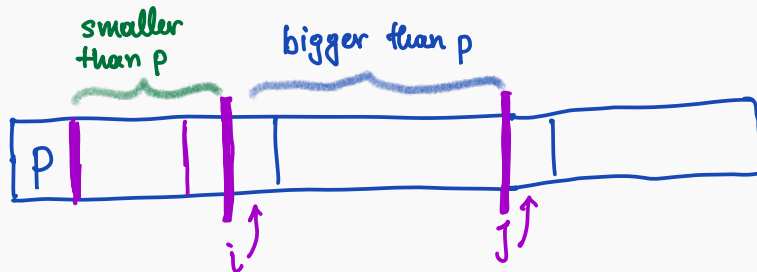


smaller bigger

it does not necessarily keep the order



Use 2 indices



already partitioned unpartitioned

We can implement the algorithm
IN PLACE
(no extra memory)

if $A[j] > A[0]$ then $j \leftarrow j+1$

if $A[i] < A[0]$ then
 swap($A[i], A[j]$)
 $i \leftarrow i+1$
 $j \leftarrow j+1$

Q: How do we use it in QSORT?

Find the k -th smallest element in an array

k -median

k -order statistic

median $k = \frac{n}{2}$

We can solve the problem in $O(n \lg n)$ (sort)

Can we do better? PARTITION

QUICKSELECT($A[1..n]$, k):

if $n = 1$

return $A[1]$

else

Choose a pivot element $A[p]$

$r \leftarrow \text{PARTITION}(A[1..n], p)$

if $k < r$

return $\text{QUICKSELECT}(A[1..r-1], k)$

else if $k > r$

return $\text{QUICKSELECT}(A[r+1..n], k-r)$

else

return $A[r]$

3	8	2	5	1	4	7	6
---	---	---	---	---	---	---	---

1	2	3	8	5	4	7	6
---	---	---	---	---	---	---	---

- 2nd elem: Search left
- 4th elem: Search right

Find the k -th smallest element in an array

k -median

k -order statistic

median $k = \frac{n}{2}$

QUICKSELECT($A[1..n]$, k):

if $n = 1$

return $A[1]$

else

Choose a pivot element $A[p]$

$r \leftarrow \text{PARTITION}(A[1..n], p)$

if $k < r$

return QUICKSELECT($A[1..r-1]$, k)

else if $k > r$

return QUICKSELECT($A[r+1..n]$, $k-r$)

else

return $A[r]$

We can solve the problem in $O(n \log n)$ (SORT)

Can we do better? PARTITION

3	8	2	5	1	4	7	6
---	---	---	---	---	---	---	---

1	2	3	8	5	4	7	6
---	---	---	---	---	---	---	---

• 2nd elem: Search left

• 4th elem: Search right

Worst case complexity $O(n^2)$

the complexity depends on the pivot

"good" pivot if $l \leq \alpha n$ for $\alpha < 1$

BUT how?

For the median, pick a random pivot

average complexity $O(n)$

Q: Can we do better?

median of medians

mom is a good pivot for PARTITION

```
MOMSELECT(A[1 .. n], k):
  if  $n \leq 25$  ⟨⟨or whatever⟩⟩
    use brute force
  else
     $m \leftarrow \lfloor n/5 \rfloor$ 
    for  $i \leftarrow 1$  to  $m$ 
       $M[i] \leftarrow \text{MEDIANOFFIVE}(A[5i - 4 .. 5i])$  ⟨⟨Brute force!⟩⟩
     $mom \leftarrow \text{MOMSELECT}(M[1 .. m], \lfloor m/2 \rfloor)$  ⟨⟨Recursion!⟩⟩
     $r \leftarrow \text{PARTITION}(A[1 .. n], mom)$ 
    if  $k < r$ 
      return  $\text{MOMSELECT}(A[1 .. r - 1], k)$  ⟨⟨Recursion!⟩⟩
    else if  $k > r$ 
      return  $\text{MOMSELECT}(A[r + 1 .. n], k - r)$  ⟨⟨Recursion!⟩⟩
    else
      return  $mom$ 
```

[Blum, Floyd, Pratt, Rivest, Tarjan : 1973]

T T Sun T T

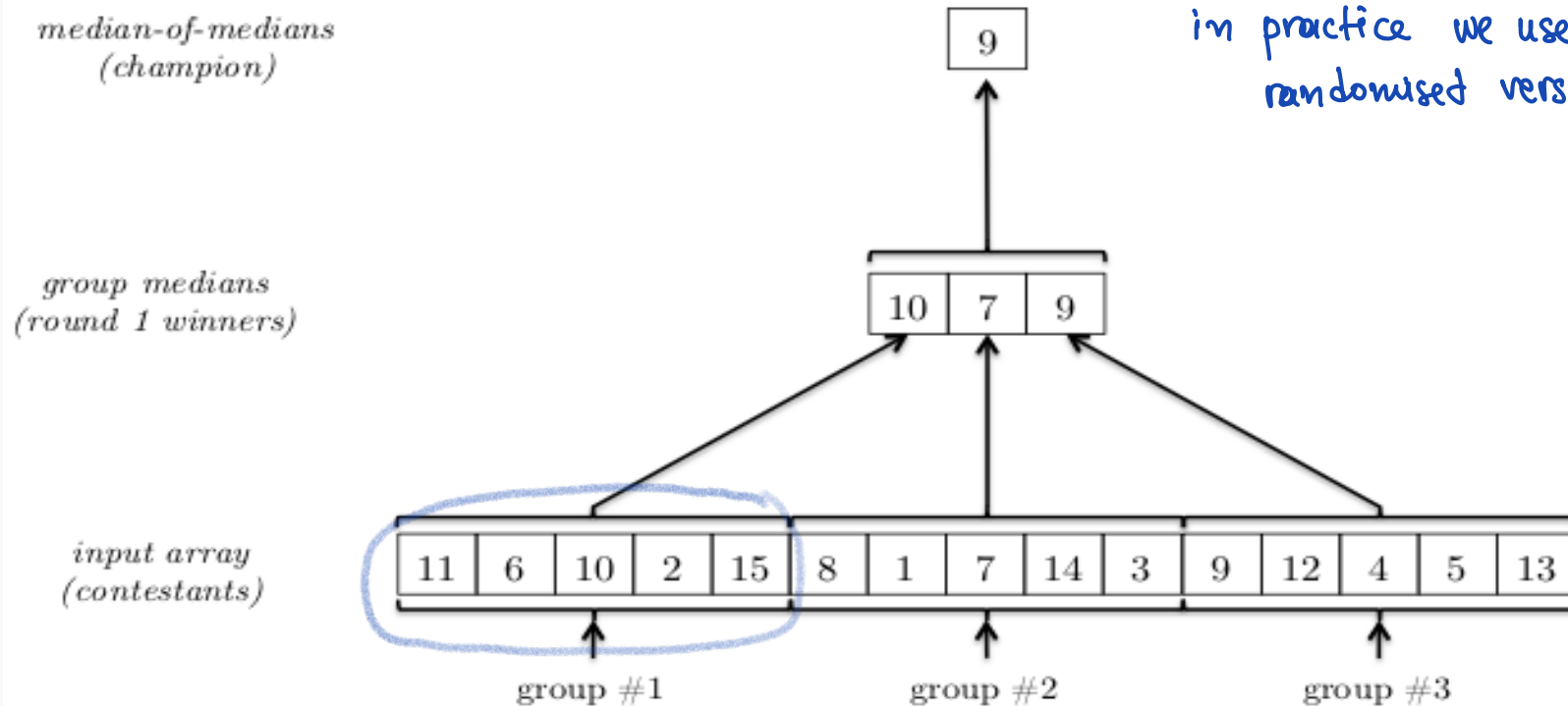
median of medians (an example)

Divide and Conquer

Complexity $O(n)$ deterministic

the constant is big(6 or 7)

in practice we use the randomised version



the analysis is not straightforward
the magic number is 5
for the blocks

Sort :: QuickSort

Divide & Conquer
[Hoare] Turing award

QSORT $O(n \lg n)$ average case

MERGESORT $O(n \lg n)$ worst case

```
QUICKSORT( $A[1..n]$ ):  
  if ( $n > 1$ )  
    Choose a pivot element  $A[p]$   
     $r \leftarrow \text{PARTITION}(A, p)$   
    QUICKSORT( $A[1..r-1]$ )  «Recurse!»  
    QUICKSORT( $A[r+1..n]$ ) «Recurse!»
```

```
PARTITION( $A[1..n], p$ ):  
  swap  $A[p] \leftrightarrow A[n]$   
   $\ell \leftarrow 0$  «#items < pivot»  
  for  $i \leftarrow 1$  to  $n-1$   
    if  $A[i] < A[n]$   
       $\ell \leftarrow \ell + 1$   
      swap  $A[\ell] \leftrightarrow A[i]$   
  swap  $A[n] \leftrightarrow A[\ell + 1]$   
  return  $\ell + 1$ 
```

Complexity $O(n \lg n)$ on the average

Advantages: (i) small constants in $O(\cdot)$
(ii) In place

Q: Can we do better?
when? how? why? Example?

A question about the $\mathcal{O}$ notation



What is the complexity of the following program?

```
for (int i = 0; i < n; ++i) {  
    for (int j = 0; j < sqrt(n); ++j) {  
        cout << j << endl;  
    }  
}
```

- (1) $\mathcal{O}(n)$
- (2) $\mathcal{O}(n\sqrt{n})$
- (3) $\mathcal{O}(n^2)$
- (4) $\mathcal{O}(n \log(n))$

A question about the $\mathcal{O}$ notation



What is the complexity of the following program?

```
for (int i = 0; i < n; ++i) {  
    for (int j = 0; j < sqrt(n); ++j) {  
        cout << j << endl;  
    }  
}
```

what if I change it
to $\sqrt{i}$?

$$T(n-1) = \sum_{i=1}^{n-1} \sqrt{i}$$

$$T(n) = \sum_{i=1}^n \sqrt{i}$$

$$T(n) - T(n-1) =$$

- (1) $\mathcal{O}(n)$
- (2) $\mathcal{O}(n\sqrt{n})$
- (3) $\mathcal{O}(n^2)$
- (4) $\mathcal{O}(n \log(n))$

Contents

General Info

Search in an unsorted array

Search in a sorted array

Basic algebraic operations

Multiplication

Exponentiation

Fibonacci

Additional issues

Intro to binary search

Find the first 1

$n = 12$

0	0	0	0	0	0	0	1	1	1	1	1
---	---	---	---	---	---	---	---	---	---	---	---

The binary search algorithm

We search a sorted array
somehow

$O(\lg n)$

Information theoretical
lower bound

• What is this?

• what if you search
for 2 elements

```
#include <iostream>
#include <limits>
#include <algorithm>
using namespace std;
```

```
int A[numeric_limits<int>::max()];
```

```
int main() {
    int n;
    cin >> n;
    for (int i = 0; i < n; ++i) { cin >> A[i]; }
```

```
    sort(A, A+n);
```

```
    if (std::binary_search (A, A+n, 13))
```

```
        std::cout << "found!\n";
```

```
    else
```

```
        std::cout << "not found.\n";
```

```
    return 0;
```

```
}
```

iterators
Modern version of pointers

element
to find

Binary search in unimodal array

unique max
find it



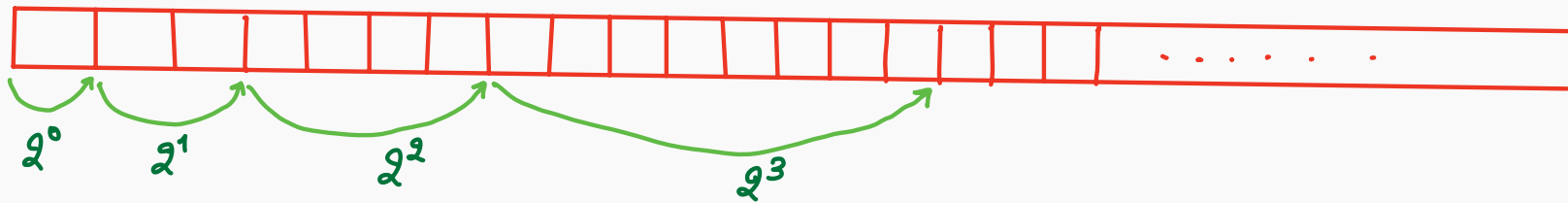
$n=14$

1	3	4	5	7	8	10	12	13	14	10	9	6	2
---	---	---	---	---	---	----	----	----	----	----	---	---	---

As the first example, but think about it

Exponential search (binary search without an upper bound)

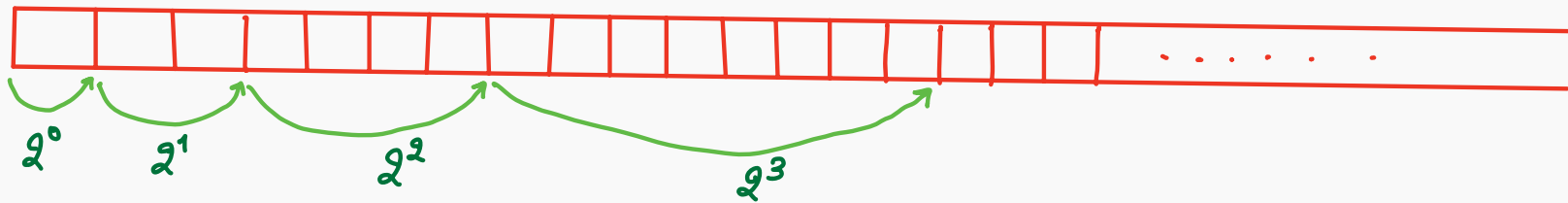
What to do if we do not know (or it is infinite)
the size of the sorted array?



Problem: # occurrences of an element in a sorted array?

Exponential search (binary search without an upper bound)

What to do if we do not know (or it is infinite)
the size of the sorted array?



Problem: # occurrences of an element in a sorted array?

find the first and the last occurrence in $O(\lg n)$

1 2 3 4 7 13 13 13 55 101 101

Problems in binary search

P1 Consider $a, b \in \mathbb{Z}$.
Compute $a \div b$ and $a \bmod b$ using binary search
Complexity? What does this complexity count?

P2 Consider a sorted array A , not necessarily with distinct elems.
Find the k -th occurrence of an element a
Find the #occurrences of an element.

1 2 2 3 3 4 4 4 4 5 5 6 7 8
the 3rd occurrence of 4 is at position 8

Problems in binary search

P3 Let A be a sorted array. Find the number of distinct elements.

$A = 1\ 1\ 1\ 2\ 2\ 2\ 2\ 3\ 4\ 7\ 7\ 7\ 8$ 1 2 3 4 7 8

∃ straightforward ALG with complexity $O(n)$

Q: Can we do better? Why?

P4. How do we do binary search in 2D arrays?

P5. What is continuous binary search?

Contents

General Info

Search in an unsorted array

Search in a sorted array

Basic algebraic operations

Multiplication

Exponentiation

Fibonacci

Additional issues

Integer Multiplication $x \cdot y$

Naive algorithm $\mathcal{O}(n^2)$, where n is the number of bits of x and y .

$$x = 2356 = 23 \cdot 10^2 + 56$$

$$y = 1478 = 14 \cdot 10^2 + 78$$

$$x \cdot y = (10^m a + b)(10^m c + d) = 10^{2m} ac + 10^m (bc + ad) + bd$$

$$[\text{Gauss}] \quad ac + bd - (a - b)(c - d) = bc + ad$$

FASTMULTIPLY(x, y, n): [Karatsuba]

if $n = 1$

return $x \cdot y$

else

$m \leftarrow \lceil n/2 \rceil$

$a \leftarrow \lfloor x/10^m \rfloor$; $b \leftarrow x \bmod 10^m$ $\langle\langle x = 10^m a + b \rangle\rangle$

$c \leftarrow \lfloor y/10^m \rfloor$; $d \leftarrow y \bmod 10^m$ $\langle\langle y = 10^m c + d \rangle\rangle$

$e \leftarrow \text{FASTMULTIPLY}(a, c, m)$

$f \leftarrow \text{FASTMULTIPLY}(b, d, m)$

$g \leftarrow \text{FASTMULTIPLY}(a - b, c - d, m)$

return $10^{2m} e + 10^m (e + f - g) + f$ } Conquer $\rightarrow$ Combine

Divide {

$$\mathcal{O}(n^{\lg_2 3}) = \mathcal{O}(n^{1.59})$$

Q: Can we do better?

(Binary) Exponentiation a^n

$$a^n = \underbrace{a \cdot a \cdot \dots \cdot a}_{n \text{ times}}$$

SLOWPOWER(a, n):

$x \leftarrow a$

for $i \leftarrow 2$ to n

$x \leftarrow x \cdot a$

return x

Complexity : $O(n)$ multiplications

arithmetic operations

Bit complexity

Q: Can we do better?

Exponentiation á la Piṅgala

Piṅgala Indian prosodist (2nd century BCE)

$$a^n = \begin{cases} 1 & \text{if } n = 0 \\ (a^{n/2})^2 & \text{if } n > 0 \text{ and } n \text{ is even} \\ (a^{\lfloor n/2 \rfloor})^2 \cdot a & \text{otherwise} \end{cases}$$

Complexity $O(\lg n)$ multiplications

How much does each multiplication cost?

PIṅGALAPOWER(a, n):

if $n = 1$

return a

else

$x \leftarrow \text{PIṅGALAPOWER}(a, \lfloor n/2 \rfloor)$

if n is even

return $x \cdot x$

else

return $x \cdot x \cdot a$

$$\begin{aligned} a^9 &= (a^{\lfloor 9/2 \rfloor})^2 \cdot a \\ &= (a^4)^2 \cdot a \\ &= ((a^2)^2)^2 \cdot a \end{aligned}$$

Q: What is the cost of 1 multiplication?

Exponentiation á la Egypt

the way (sort of) of Egyptian peasants

$$a^n = \begin{cases} 1 & \text{if } n = 0 \\ (a^2)^{n/2} & \text{if } n > 0 \text{ and } n \text{ is even} \\ (a^2)^{\lfloor n/2 \rfloor} \cdot a & \text{otherwise} \end{cases}$$

PEASANTPOWER(a, n):

if $n = 1$

return a

else if n is even

return PEASANTPOWER($a^2, n/2$)

else

return PEASANTPOWER($a^2, \lfloor n/2 \rfloor$) $\cdot a$

Complexity $O(\lg n)$ mults

$$\begin{aligned} a^9 &= (a^2)^{\lfloor 9/2 \rfloor} \cdot a \\ &= (a^2)^4 \cdot a \\ &= ((a^2)^2)^2 \cdot a \end{aligned}$$

Optimal Exponentiation

$$a^n = \begin{cases} 1 & \text{if } n = 0 \\ (a^{n/2})^2 & \text{if } n > 0 \text{ and } n \text{ is even} \\ (a^{\lfloor n/2 \rfloor})^2 \cdot a & \text{otherwise} \end{cases}$$

$O(\lg n)$ Pingala

$$a^n = \begin{cases} 1 & \text{if } n = 0 \\ (a^2)^{n/2} & \text{if } n > 0 \text{ and } n \text{ is even} \\ (a^2)^{\lfloor n/2 \rfloor} \cdot a & \text{otherwise} \end{cases}$$

$O(\lg n)$ Peasant



Is Pingala or Peasant optimal?

We cannot do better than $O(\lg n)$ why?
asymptotically

Optimal Exponentiation

$$a^n = \begin{cases} 1 & \text{if } n = 0 \\ (a^{n/2})^2 & \text{if } n > 0 \text{ and } n \text{ is even} \\ (a^{\lfloor n/2 \rfloor})^2 \cdot a & \text{otherwise} \end{cases}$$

$$a^n = \begin{cases} 1 & \text{if } n = 0 \\ (a^2)^{n/2} & \text{if } n > 0 \text{ and } n \text{ is even} \\ (a^2)^{\lfloor n/2 \rfloor} \cdot a & \text{otherwise} \end{cases}$$

Major open problem

Q: Can we compute efficiently the absolute minimum of the #multiplications that we need for any n ?



Is Pingala or Peasant optimal?

- 6 ops • Pingala: $a \rightarrow a^2 \rightarrow a^3 \rightarrow a^6 \rightarrow a^7 \rightarrow a^{14} \rightarrow a^{15}$
- 6 ops • Peasant: $a \rightarrow a^2 \rightarrow a^4 \rightarrow a^8 \rightarrow a^{12} \rightarrow a^{14} \rightarrow a^{15}$
- 5 ops • Optimal: $a \rightarrow a^2 \rightarrow a^3 \rightarrow a^5 \rightarrow a^{10} \rightarrow a^{15}$

The Fibonacci numbers

There is a recursive definition of the Fibonacci sequence:

$$F_1 = 1$$

$$F_2 = 1$$

$$F_n = F_{n-1} + F_{n-2}$$

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610, 987, 1597, 2584, ...

Calculate the Fibonacci numbers

memoization : remember the previous values

RECFIBO(n):

```
if  $n = 0$ 
    return 0
else if  $n = 1$ 
    return 1
else
    return RECFIBO( $n - 1$ ) + RECFIBO( $n - 2$ )
```

MEMFIBO(n):

```
if  $n = 0$ 
    return 0
else if  $n = 1$ 
    return 1
else
    if  $F[n]$  is undefined
         $F[n] \leftarrow \text{MEMFIBO}(n - 1) + \text{MEMFIBO}(n - 2)$ 
    return  $F[n]$ 
```

Complexity $2F_n - 1$

twice as much as just
counting up to F_n

ITERFIBO(n):

```
 $F[0] \leftarrow 0$   
 $F[1] \leftarrow 1$   
for  $i \leftarrow 2$  to  $n$   
     $F[i] \leftarrow F[i - 1] + F[i - 2]$   
return  $F[n]$ 
```

iterative
version

$\rightarrow O(n)$ additions

Exponential improvement
wrt RECFIBO

Q: Can we do better?

Fibonacci number as vectors

We can represent the Fibonacci sequence using vectors and a matrix

$$\begin{pmatrix} F_{n+2} \\ F_{n+1} \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} F_{n+1} \\ F_n \end{pmatrix}$$

Fibonacci number as vectors

We can represent the Fibonacci sequence using vectors and a matrix

$$\begin{pmatrix} F_{n+2} \\ F_{n+1} \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} F_{n+1} \\ F_n \end{pmatrix}$$

$$\begin{pmatrix} F_2 \\ F_1 \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} F_1 \\ F_0 \end{pmatrix}$$

$$\begin{pmatrix} F_3 \\ F_2 \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} F_2 \\ F_1 \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} F_1 \\ F_0 \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^2 \begin{pmatrix} F_1 \\ F_0 \end{pmatrix}$$

Superfast Fibonacci

$$\begin{pmatrix} F_{n+2} \\ F_{n+1} \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^n \begin{pmatrix} F_1 \\ F_0 \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^n \begin{pmatrix} 1 \\ 0 \end{pmatrix}$$

Or using only the matrix

$$\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^n = \begin{pmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{pmatrix}$$

How do we calculate the matrix power?

Superfast Fibonacci

$$\begin{pmatrix} F_{n+2} \\ F_{n+1} \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^n \begin{pmatrix} F_1 \\ F_0 \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^n \begin{pmatrix} 1 \\ 0 \end{pmatrix}$$

Or using only the matrix

$$\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^n = \begin{pmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{pmatrix}$$

How do we calculate the matrix power? Fast Exponentiation

Complexity $O(\lg n)$ #ops

⚠ Bit complexity of matrix multiplication

Generalizations?